

Truthprint: Designing an Invariant-Constrained Semantic Intermediate Representation for Provenance Watermarking

Geunsik Lim

Abstract—Token-level watermarks for large language models (LLMs) are fragile under translation and paraphrasing because their signal lives in surface token choices. This paper proposes *Truthprint*, a semantic intermediate representation (IR) that separates *locked invariants* (polarity, quantity, time, attribution, roles) from *controlled realization variables* through a machine-checkable, typed mutability contract. We evaluate two provenance mechanisms that share this IR and are careful to distinguish them. (i) A *realization-carrier* scheme embeds an authenticated, error-corrected payload in realization freedom; against real neural machine translation this is an *honest negative*—the Stage-1 surface parser recovers 0/192 NLLB outputs—because translation rewrites the carriers. (ii) A *meaning-digest* scheme instead registers a keyed tag over the invariant digest in an out-of-band ledger and re-verifies it after transformation, carrying no realization payload. On 4,800 real NLLB translations across six languages, a Stage-2 lexicon-based extractor recovers invariant fields at 0.90–1.00 and the meaning-digest scheme authenticates translated documents cross-lingually (1.000 document-level attribution for five of six languages; Arabic weaker) while rejecting single-field tampering. We prove three *conditional* properties (protected-field fidelity under a sound validator, fresh-payload unforgeability reducing to the MAC, and a per-key-nonce tag-acceptance bound 2^{-7}), and ship a seeded, self-testing cryptographic/coding core (zero authenticated false positives in 20,000 trials; 95% upper bound 1.5×10^{-4}). On the same real-MT pipeline, a real KGW token watermark collapses under translation (TPR@1%FPR 0.96 clean to 0.00–0.38 translated) while the meaning-digest scheme survives (0.50–0.98), a real single-baseline comparison. At a *matched* 1% FPR on real LLM-MT, a real embedding-region baseline (SemStamp) survives round-trip (1.00) but degrades cross-lingually (0.94 Korean to 0.38 Arabic) as embedding buckets drift, while the typed meaning-digest survives all conditions and, unlike similarity/robust hashing, rejects single-field tampers (typed 1.00 vs. SimHash/MinHash 0.00). All results are closed-domain and small-scale; wide-coverage neural parsing, broader official baselines (SynthID, SemStamp, SWAN), and adaptive attacks remain necessary before a general comparative-robustness claim. Truthprint does not identify arbitrary AI text; it verifies consistency with a participating keyed generator.

Index Terms—Intermediate representation, LLM watermarking, paraphrase robustness, provenance, semantic watermarking, SynthID-Text, translation robustness

I. INTRODUCTION

Large language models can generate fluent text at scale, creating a growing need for provenance mechanisms that survive ordinary content transformation. Generation-time watermarking modifies the decoding process so that an authorized detector can identify a statistical signature in generated text.

SynthID-Text demonstrated that such watermarking can be integrated into sampling without retraining the underlying LLM and can be detected without rerunning the generator [1].

Regulation also motivates work on machine-readable marking. Article 50 of the EU Artificial Intelligence Act (Regulation (EU) 2024/1689) introduces transparency obligations concerning synthetic outputs, including text [3]. Truthprint is a research design, however, and we neither interpret the regulation nor claim that the proposed mechanism satisfies it; robustness to meaning-preserving transformation is only a technical objective here [4]. Reliable provenance also matters for data hygiene: training on unattributed synthetic text can degrade model quality over successive generations, a failure mode documented at scale in *Nature* [2]. Truthprint targets this gap with an authenticated design whose cross-lingual robustness remains to be established empirically.

Token-level methods, however, embed their signal in surface token choices. Translation, paraphrasing, sentence restructuring, and summarization replace many of those choices while preserving substantial meaning. This creates a structural mismatch: the desired provenance claim concerns the persistence of content identity, whereas the carrier often depends on language-specific lexical decisions.

Recent semantic watermarking methods reduce this dependence by using sentence embeddings, contextual semantic representations, or structured semantic graphs [15], [16], [17], [18], [19]. Yet semantic similarity alone does not explicitly distinguish meaning components that must never change from linguistic choices that may safely vary. Two sentences may be close in embedding space while differing in negation, quantity, temporal scope, attribution, modality, or causal direction.

This paper proposes *Truthprint*, a watermark-specific semantic IR that explicitly separates:

- 1) **semantic invariants**, which must remain fixed; and
- 2) **controlled realization variables**, which may encode watermark symbols only when semantic and pragmatic equivalence is validated.

The central novelty is therefore not the invention of a new surface language. It is a machine-checkable contract defining which transformations are permitted to carry a watermark and which semantic fields are locked. Figure 1 shows the end-to-end data flow.

We can state the delta over the closest prior work precisely. Embedding-based semantic watermarks [15], [16], [17], [19] define admissibility through a *metric*: two texts are treated as equivalent when their representations are close under a learned

distance. Truthprint instead defines admissibility through a *typed predicate*: a transformation is admissible only when a symbolic invariant relation holds field by field. A watermark symbol then ranges not over tokens or embedding regions but over the key-dependent equivalence classes induced by that relation. This has two consequences that motivate the analysis in Section IX. First, conditional on sound extraction and validation, encoding only moves within an invariant-equivalence class and therefore preserves the protected fields. Second, because authenticated attribution is gated by a payload bound to the invariant digest rather than by a similarity score, fresh-payload forgery reduces to the security of the message-authentication code. This statement does not cover the fallback statistical detector, whose system-level false-positive rate still requires held-out calibration. Structured-semantics watermarking such as SWAN [18] shares the use of a graph carrier but adopts an existing meaning formalism wholesale; Truthprint instead types each field with an explicit mutability policy and validates every realization against the resulting contract before release.

A. Contributions

This work makes the following contributions:

- **Method.** We formulate watermark carriers as secret-keyed realization choices constrained by a field-level mutability contract (*locked*, *conditional*, or *realization-only*), and combine uncertain-carrier erasures with error correction and invariant-bound payload authentication.
- **Conditional analysis.** Assuming sound invariant extraction and validation, we establish protected-field fidelity; under standard cryptographic assumptions, we relate fresh-payload forgery and per-test tag acceptance to the message-authentication code. We also state the finite-code rank condition rather than presenting the observed erasure transition as a universal threshold.
- **Stage-1 evidence and falsifiable protocol.** A self-contained implementation verifies the coding and cryptographic core, while a closed-domain English realization demonstrates composition on sentence strings. We explicitly separate these measured results from a diagnostic simulation and specify the multilingual, official-baseline evaluation needed to test—not presume—translation and paraphrase robustness.

B. Claim–Evidence Boundary

To prevent the design goal from being mistaken for an empirical result, Table I separates what the present artifact demonstrates, what the analysis proves only under explicit assumptions, and what remains a hypothesis. In particular, “language-neutral” describes the intended schema interface rather than a measured property of the current English template frontend.

II. BACKGROUND AND RELATED WORK

A. Regulatory Context

Text watermarking is relevant to emerging transparency regulation, but no watermark design alone establishes legal

compliance. Under Article 50 of the EU AI Act, providers of generative systems must mark synthetic outputs in a machine-readable, detectable format [3]; the accompanying policy discussion stresses that a mark is only useful if it is *robust*, since a signal that dissolves under paraphrasing or translation cannot support reliable downstream attribution [4]. This motivates the technical problem addressed here without treating robustness as a legal conclusion. Truthprint is proposed as a possible semantic layer complementary to token-level marks, not as a compliance product.

B. SynthID-Text

Dathathri et al. introduced SynthID-Text as a scalable watermarking system for LLM outputs [1]. The method modifies the sampling process rather than model training and uses a keyed scoring mechanism together with tournament sampling. Its design includes a pseudorandom score generator, a watermark-aware sampler, and a detector that aggregates token-level evidence. The original work considered both non-distortionary and distortionary operating regimes and also addressed speculative sampling for production throughput.

The reference implementation exposes a training-free weighted-mean detector and a stronger learned Bayesian detector [5]. Hugging Face Transformers subsequently integrated a SynthID text-watermarking configuration into its generation stack [6]. These implementations make SynthID-Text a useful operational baseline.

Its principal limitation for the present problem is that the watermark carrier remains tied to token selections. Meaning-preserving transformations can replace those token decisions while keeping the main proposition intact.

C. Theoretical Analysis of SynthID-Text

Omidi et al. presented a theoretical and empirical analysis of SynthID-Text, focusing on tournament depth and detector behavior [7]. Their work shows that the mean-score detector can become vulnerable under layer inflation, whereas the Bayesian detector is more robust to variation in tournament layers. The analysis also identifies the Bernoulli parameter $p = 0.5$ as optimal under the studied assumptions.

This line of work demonstrates that watermark robustness depends not only on how a signal is embedded but also on the detector’s statistical assumptions. Truthprint therefore does not rely on a single uncorrected aggregate score. It uses structured carrier recovery, confidence-weighted erasures, error-correcting decoding, authenticated payload verification, and a calibrated statistical fallback.

D. Robustness Assessment and Hybrid Defenses

Han et al. evaluated SynthID-Text under paraphrasing, copy-and-paste modification, synonym substitution, and back-translation [8]. They reported significant degradation under meaning-preserving transformations and proposed SynGuard, a hybrid system combining SynthID-style probabilistic masking with semantic alignment inspired by semantic-invariant watermarking. Their reported experiments showed an average

TABLE I
CLAIM–EVIDENCE MAP FOR THE PRESENT SUBMISSION. “NOT ESTABLISHED” MEANS THAT THE CLAIM MUST NOT BE INFERRED FROM THE DIAGNOSTIC SIMULATION.

Claim	Status	Supporting evidence or missing test
Coding/authentication round trip	Demonstrated	Deterministic core tests P1–P4 and closed-domain string tests L1–L3.
Protected-field fidelity	Conditional (validator soundness measured)	Proposition 1, assuming a sound invariant extractor and validator. The soundness assumption is measured as the validator’s false-accept rate on meaning-altering edits: 0 observed on the closed domain (C1) and ≤ 0.01 under real MT (Sec. X-F); wide-coverage soundness remains open.
Fresh-payload unforgeability	Conditional	Reduction to EUF-CMA security of the MAC; replay and nonce discovery are separate deployment concerns.
Surface-carrier translation robustness	Refuted (real MT)	Stage-1 surface parser recovers 0/192 real NLLB translations (Sec. X-C).
Meaning-layer recovery under real MT	Demonstrated (closed-domain, six languages)	Stage-2 extractor recovers fields at 0.90–1.00 over 4,800 real NLLB translations in six languages, document attribution 1.000 (CI [0.929, 1.000]) for five of six languages, and meaning-based authentication attributes cross-lingually (Secs. X-D, X-F); wide-coverage parsing and larger document counts remain open.
Wide-coverage neural parsing on open-domain text	Pilot (positive on open domain)	In a small LLM-MT pilot (96 sentences, six conditions) a neural extractor beats the lexicon on open-domain wording (0.614 vs. 0.481 field recovery) while the lexicon leads on closed templates; single-model and small-scale caveats apply (Sec. X-E).
Paraphrase/adaptive-attack robustness	Not established	Requires meaning-preserving attack outputs, held-out detector calibration, and ValidRemoval evaluation.
System-level false-positive rate	Not established	The tag bound is per tested fresh candidate; parser search, tested key–nonce pairs, and fallback scores require end-to-end calibration.
Semantic-level false-positive rate (contract collision)	Measured (closed-domain)	The 2^{-7} bound covers cryptographic tag collision only; the meaning-digest floor is the contract-collision probability. On the closed-domain corpus this floor is 5.2×10^{-3} ($\approx 1/192$) for the six-field contract—matching the measured 0.001–0.010 real-MT false positives and $\sim 2 \times 10^7 \times$ the 2^{-32} bound (Sec. X-F). Open-domain text has lower contract entropy, so estimating it per corpus is a deployment prerequisite.
Comparative robustness vs. token and semantic baselines	Demonstrated (real MT: KGW token; matched-FPR SemStamp + robust hashing)	On the same pipeline a real KGW watermark collapses under real NLLB translation (TPR@1%FPR 0.960 clean $\rightarrow$ 0.000–0.380 translated) while the meaning-digest scheme survives (0.502–0.980); Table IV, Sec. X-G. At a matched 1% FPR a real embedding-region baseline (SemStamp) survives round-trip but degrades cross-lingually (0.937 KO $\rightarrow$ 0.377 AR) while the typed scheme survives, and only the typed contract rejects single-field tampers (Table V, Sec. X-H). Official implementations (SynthID, SWAN) remain future work.
Distinction from robust/semantic hashing	Demonstrated (closed-domain, real MT)	Robust hashes (exact, MinHash, SimHash) either die under translation or cannot localize a typed field change (reject 0% of single-field tampers vs. typed 1.000); the meaning-digest is a typed, keyed member of the family with an analyzed collision floor (Secs. II-J, X-H).
Translator-independent (separated) frontend	Pilot (open-domain positive on one field)	A model-separated embedding frontend recovers open-domain temporal direction (0.944) where the lexicon abstains (0.000); static embeddings remain insufficient for the full contract, motivating the separated-backend LLM frontend (Sec. X-E).

F1 improvement of 11.1 percentage points over SynthID-Text across the evaluated attacks.

SynGuard supports the same broad motivation as Truthprint: semantic information can improve robustness after surface tokens change. The distinction is that Truthprint makes semantic preservation explicit and field-specific. A candidate is rejected if it changes a protected property such as negation, number, attribution, causal direction, or temporal relation, even when a global embedding similarity remains high.

E. Semantic and Structured Watermarking

SemStamp partitions sentence-embedding space using locality-sensitive hashing and selects sentence candidates that fall into keyed semantic regions [15]. SemaMark conditions watermark behavior on contextual semantic representations [16], while semantic-invariant watermarking similarly uses

semantic context to stabilize watermark decisions under paraphrasing [17].

These methods reduce dependence on exact token histories, but embedding geometry does not directly expose which critical fact changed. High semantic similarity may coexist with altered polarity, quantity, attribution, modality, or causal direction.

SWAN embeds watermark information into Abstract Meaning Representation (AMR) structures [18]. This is closely related to Truthprint because both use structured semantics as a carrier. Truthprint extends that direction by defining a watermark-specific IR, assigning a mutability policy to each field, incorporating discourse and provenance metadata, and validating every realization against the immutable semantic contract before release.

Dual-semantic-embedding watermarking combines contextual and token-level semantic information to improve trans-

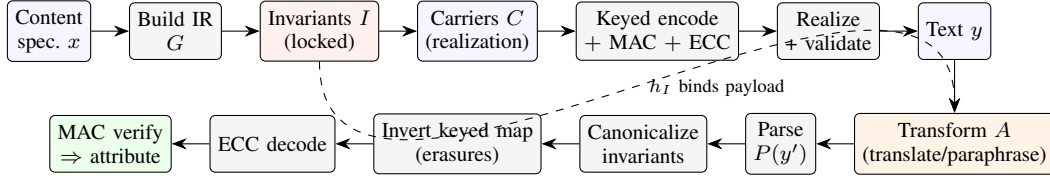


Fig. 1. Truthprint data flow. Encoding (top, left→right) locks the invariant set I , maps an authenticated, error-corrected payload onto realization carriers C , and validates every candidate against the contract before emitting text. Detection (bottom, right→left) re-parses possibly transformed text, canonicalizes invariants, inverts the keyed map with erasures, decodes, and attributes only on MAC verification. The invariant digest h_I binds the payload, so an invariant-altering transform breaks attribution (Theorem 2).

lation and paraphrase robustness [19]. The difficulty is fundamental: He et al. show that token-level watermarks lack cross-lingual consistency and can be removed by translating through a pivot language [20]. Truthprint instead seeks to make language invariance explicit by mapping all supported languages into one canonical semantic representation.

F. Black-Box Watermark Identification

Gloaguen et al. studied whether watermark deployment can be identified through limited black-box queries [22]. Their tests cover Red–Green, fixed-sampling, and cache-augmented watermark families. The broader security implication is that the existence or family of a watermark may be externally distinguishable even when the key is secret.

Truthprint therefore adopts a Kerckhoffs-style threat model: the attacker may know the schema, carrier families, encoder design, and detector algorithm. Security should depend on the secret key, document nonce, keyed assignment, and authenticated payload rather than on obscurity of the scheme.

G. Scrubbing, Spoofing, and Distillation

Yi et al. investigated watermark scrubbing and spoofing in unauthorized knowledge distillation [24]. Their contrastive-decoding-guided framework addresses both removing inherited watermark traces and falsely reproducing them. A related line shows that watermarks are *radioactive*: statistical traces survive into models fine-tuned on watermarked outputs, enabling detection of unauthorized training [23]. This attack setting operates at the model level rather than on one transformed document, but it highlights the importance of forgery resistance.

Truthprint authenticates payloads using a message authentication code (MAC). Statistical carrier agreement alone is insufficient for strong attribution; a payload is accepted as authentic only when its MAC verifies.

H. Adversarial Paraphrasing

Cheng et al. proposed detector-guided adversarial paraphrasing as a training-free attack across neural detectors, watermarking systems, and zero-shot AI-text detectors [25]. Because a continuous detector score can serve as an optimization signal, Truthprint recommends coarse or access-controlled detector interfaces, heterogeneous carrier families, and joint evaluation of watermark removal and semantic preservation.

A valid removal attack should satisfy both conditions:

$$\text{ValidRemoval} = \mathbb{I} \left[D_K(y') < \tau_D \wedge \text{InvariantEq}(P(y), P(y')) = 1 \right]. \quad (1)$$

This prevents meaning-destroying rewrites from being counted as successful semantic-preserving attacks.

I. Provable and Cryptographic Watermarking

The token-level lineage begins with the green–red list watermark of Kirchenbauer et al. [9], which SynthID-Text generalizes through tournament sampling. A parallel line seeks formal guarantees: Zhao et al. prove robustness for a fixed-grouping unigram watermark [10], Kudithipudi et al. give distortion-free watermarks robust to editing [11], and Christ et al. introduce cryptographically *undetectable* watermarks whose presence is indistinguishable without the key [12]. Closest in aim, Liu et al. propose an unforgeable publicly verifiable watermark that separates generation and detection keys [14], and Fernandez et al. consolidate statistical tests with guarantees that hold at very low false-positive rates [13]. Truthprint inherits the spirit of this line at the provenance layer rather than the token layer: fresh-payload unforgeability and per-candidate tag acceptance (Theorems 1 and 2) reduce to standard cryptographic assumptions on the MAC. These are not guarantees for the fallback score or the end-to-end detector, both of which require empirical calibration. Truthprint does not claim undetectability; its aim is authenticated attribution designed to retain evidence under some transformations, a property not established by the present Stage-1 evaluation.

J. Robust and Semantic Hashing

Registering a meaning-digest tag in a ledger and re-verifying it after transformation resembles *robust hashing*: perceptual or semantic fingerprints that map near-duplicate content to near-identical digests for copy detection and deduplication, from MinHash [27] and SimHash [28] to learned semantic hashing. The difference is *what* is hashed. Robust hashing answers “is this the same string, the same token set, or a nearby point in embedding space?”—a *similarity* question whose tolerance both admits false matches on low-entropy content and, decisively, *cannot localize which typed fact changed*: a single-field meaning-altering edit stays within tolerance, which we measure directly in Section X-H (SimHash and MinHash reject 0% of such edits). Truthprint instead hashes a *typed invariant contract* and authenticates with a keyed MAC, so a change to any contract field breaks verification while realization-only variation and translation do not. The meaning-digest scheme is thus a *typed, keyed* member of this family whose false-positive floor is the contract-collision probability (Sec. X-F)—analyzed here rather than left implicit as in generic robust hashing.

K. Positioning of Truthprint

Table II summarizes the principal carrier used by representative systems.

III. PROBLEM DEFINITION

Let x be a semantic content specification, K a secret key, m a provenance payload, E_K the encoder, R_ℓ a realizer for language ℓ , A a transformation, P a multilingual parser, and D_K the detector. The generated text is

$$y = R_\ell(E_K(x, m)), \quad (2)$$

and an observed transformed text is

$$y' = A(y). \quad (3)$$

The detector returns

$$(s, \hat{m}, c) = D_K(P(y')), \quad (4)$$

where s is a detection score, $\hat{m}$ a recovered payload or payload prefix, and c a confidence report.

The design targets four objectives:

- 1) **Semantic fidelity**: $I(P(y)) = I(x)$ for invariant extractor I .
- 2) **Natural-language quality**: fluency, relevance, and style remain above deployment thresholds.
- 3) **Robust detection**: detection remains reliable under transformations preserving sufficient invariant and carrier evidence.
- 4) **Controlled authenticated false positives**: under the stated PRF and independence assumptions, an independently produced decoded payload rarely passes the keyed tag check; the statistical fallback requires separate empirical calibration.

IV. THREAT MODEL

A. Benign Transformations

The detector should tolerate Unicode normalization, punctuation changes, spelling correction, translation, round-trip translation, synonym substitution, active–passive alternation, clause splitting and merging, sentence-level paraphrasing, moderate paragraph rewriting, and limited insertion or deletion.

B. Non-Adaptive and Adaptive Attackers

A non-adaptive attacker applies off-the-shelf translation, paraphrasing, or summarization without knowing K . An adaptive attacker may know the schema and detector design, repeatedly rewrite text, splice documents, target known carrier families, or query an exposed detector.

C. Out-of-Scope Claim

Truthprint does not identify all AI-generated text. It verifies consistency with a participating keyed generator. Absence of a signal does not prove human authorship.

V. TRUTHPRINT INTERMEDIATE REPRESENTATION

A. Three-Layer Structure

A Truthprint document contains three conceptual layers.

Layer A: Semantic invariants. Entities, predicates, semantic roles, polarity, negation scope, modality, quantities, temporal intervals, causal direction, conditions, exceptions, attribution, quotation status, and uncertainty.

Layer B: Controlled realization variables. Voice, event/s-tate framing, clause packaging, discourse-connective family, proposition ordering, nominal/verbal realization, anaphoric choice, and optional explanatory apposition.

Layer C: Provenance and coding metadata. Scheme identifier, key identifier, payload, nonce, error-correcting-code parameters, carrier assignments, reliability estimates, invariant digest, and optional signed server-side provenance.

B. Typed Mutability

Each field f is assigned

$$\text{Mutability}(f) \in \{\text{locked}, \text{conditional}, \text{realization-only}\}. \quad (5)$$

Locked fields must never change. Conditional fields may change only when context-specific validators approve. Realization-only fields may be used as watermark carriers.

C. Example Representation

Listing 1. Simplified Truthprint representation.

```
{
  "entities": [
    {"id": "e1", "type": "PERSON_ROLE",
     "canonical": "developer"},
    {"id": "e2", "type": "TECHNICAL_ERROR",
     "canonical": "server_error"}
  ],
  "events": [
    {"id": "v1", "predicate": "FIX",
     "roles": {"agent": "e1", "patient": "e2"},
```

TABLE II
POSITIONING OF TRUTHPRINT RELATIVE TO REPRESENTATIVE WATERMARKING AND ATTACK RESEARCH.

Category	Representative work	Main carrier or focus	Limitation addressed by Truthprint
Token-level sampling	SynthID-Text	Keyed token-selection statistics	Translation and paraphrasing replace token decisions.
Sentence-space watermarking	SemStamp	Keyed sentence-embedding regions	Embedding regions are not typed by invariant category.
Semantic-context watermarking	SIR, SemaMark	Contextual semantic embeddings	Similarity does not explicitly protect negation, numbers, causality, or attribution.
Hybrid defense	SynGuard	Token watermark plus semantic alignment	Semantic constraints are less explicit than a typed IR contract.
Structured semantics	SWAN	AMR-level structures	Existing semantic form is not a complete watermark-specific mutability contract.
Dual embeddings	DEW	Context and token embeddings	Language invariance remains implicit rather than represented as a canonical IR.
Attack analysis	Layer inflation, black-box probing, adversarial paraphrasing	Detector assumptions and adaptive transformation	Motivates authenticated payloads, conservative interfaces, and erasure-aware decoding.
Proposed system	Truthprint	Typed invariants and controlled realization classes	Explicitly specifies what must remain fixed and what may carry symbols.

```

    "polarity": "positive",
    "time": { "relation_to_creation": "previous_day" }
  },
  "invariants": {
    "locked_paths": [
      "/events/*/predicate",
      "/events/*/roles",
      "/events/*/polarity",
      "/events/*/time"
    ]
  },
  "realization_space": {
    "voice_vl": ["active", "passive"],
    "event_framing": ["verbal", "nominal"]
  }
}

```

D. Worked Example

Consider the fact with locked invariants `FIX(agent=developer, patient=server_error)`, polarity positive, time *previous_day*. Two binary carriers are available: `VOICE` $\in$ {active, passive} and `TIME-POS` $\in$ {front, end}. Suppose the keyed map assigns the code symbols so that this unit must realize as passive with the time phrase at the front:

“On the previous day, the server error was fixed by the developer.”

A benign transformation—say an English→Korean translation—typically preserves the invariants but may not preserve every carrier. If translation canonicalizes voice (Korean commonly renders this actively), the `VOICE` carrier is destroyed; the detector, on re-parsing, flags it as *low confidence* and records an *erasure* rather than reading a wrong bit. The `TIME-POS` carrier and the carriers of neighboring sentences survive. Because the payload is spread across sentences and protected by an error-correcting code, the authenticated payload is still recovered once the surviving carriers exceed the capacity floor of Eq. (19). In contrast, a rewrite that inserts a negation (“was *not* fixed”) alters a locked field, changes the invariant digest h_I , and correctly fails MAC verification. This example is executed end-to-end on sentence strings in the reference implementation (Section X, properties L1–L3).

VI. FORMAL SEMANTIC CONTRACT

Let a Truthprint graph be

$$G = (V_E, V_V, E_R, E_D, \Phi), \quad (6)$$

where V_E denotes entity nodes, V_V event/state nodes, E_R semantic-role edges, E_D discourse/logical relations, and Φ scoped features such as negation, modality, time, quantity, and attribution.

We divide features into

$$G = I \cup C \cup U, \quad (7)$$

where I is the invariant set, C the controlled carrier set, and U unconstrained style not used for watermarking.

A transformed structure G' is admissible only when

$$\text{InvariantEq}(I(G), I(G')) = 1 \quad (8)$$

and

$$Q(R_\ell(G')) \geq \tau_Q, \quad (9)$$

where Q is a quality and contextual-consistency function.

Hard invariants are validated symbolically where possible. Soft invariants, such as emphasis or pragmatic implication, require bidirectional entailment, contradiction checks, and human-calibrated thresholds.

VII. WATERMARK ENCODING

A. Payload Construction

The raw payload may contain a generator-family identifier, deployment identifier, coarse time bucket, document nonce, and policy identifier. Personal identifiers, prompts, and exact timestamps should not be embedded.

The authenticated payload is

$$p = m \parallel \text{MAC}_K(m \parallel n \parallel h_I), \quad (10)$$

where n is a nonce and h_I the canonical invariant digest. The payload is then encoded using an error-correcting code, following prior use of error correction for provably robust multi-bit text watermarking [21]:

$$c = \text{ECC}(p). \quad (11)$$

B. Carrier Discovery

The encoder identifies admissible carrier families per sentence or discourse unit, for example voice, clause packaging, discourse connective family, nominal/verbal realization, and order of independent propositions. Carriers are context dependent: passive voice is rejected if it obscures an important agent.

C. Keyed Symbol Assignment

For carrier j with alternatives

$$A_j = \{a_{j,0}, \dots, a_{j,r-1}\}, \quad (12)$$

a pseudorandom permutation is generated as

$$\pi_j = \text{PRP}_K(\text{CanonicalHash}(I, G, j, n)). \quad (13)$$

The code symbol c_j selects

$$a_j^* = \pi_j(c_j \bmod |A_j|). \quad (14)$$

Because the mapping depends on the invariant graph and nonce, a global rule such as “passive means 1” does not exist.

D. Candidate Generation and Validation

For each selected alternative, the realizer generates candidates. Every candidate is reparsed and checked for hard-invariant equality, bidirectional entailment, contradiction probability, numerical consistency, entity consistency, fluency, style, and expected carrier recoverability. If none is valid, the carrier is recorded as an erasure rather than forcing degraded text.

Listing 2. Truthprint encoding pseudocode.

```
G <- BuildTruthprint(x)
I <- ExtractInvariants(G)
p <- AuthenticatePayload(K, m, nonce, Hash(I))
c <- ErrorCorrectingEncode(p)
C <- DiscoverAdmissibleCarriers(G)
for assignment in KeyedAssign(K, I, C, c):
  G2 <- ApplyCarrier(G, assignment)
  candidates <- RealizeCandidates(G2, language)
  accepted <- ValidateAgainstContract(candidates, I)
  if accepted is empty:
    MarkErasure(assignment)
  else:
    CommitBestCandidate(accepted)
return AssembleDocument()
```

VIII. WATERMARK DETECTION

Detection segments the document, identifies language, parses each segment into Truthprint, resolves entities, normalizes predicates and roles, canonicalizes quantities and time, and extracts observable carrier alternatives.

For observed carrier a_j^{obs} , the detector reconstructs the keyed permutation and derives

$$\hat{c}_j = \pi_j^{-1}(a_j^{obs}). \quad (15)$$

Uncertain observations are erasures. An ECC decoder reconstructs candidate payloads, which are accepted only when their MAC verifies.

When complete recovery is impossible, a calibrated statistical score can still be computed. For binary carriers with n

reliable observations and k keyed matches, a simple reference statistic is

$$z = \frac{k - 0.5n}{\sqrt{0.25n}}. \quad (16)$$

Production calibration must account for dependence among carrier observations.

Listing 3. Truthprint detection pseudocode.

```
segments <- SegmentAndIdentifyLanguage(text)
observations <- []
for segment in segments:
  G, conf <- ParseTruthprint(segment)
  I <- CanonicalizeInvariants(G)
  for carrier in ExtractObservableCarriers(G):
    if Reliability(carrier, conf) >= threshold:
      observations.append(InvertKeyedMap(K, I, carrier))
    else:
      observations.append(ERASURE)
candidates <- ECCDecode(observations)
valid <- VerifyMACs(K, candidates)
score <- CalibratedStatisticalScore(observations, K)
return score, Best(valid), ConfidenceReport()
```

IX. FORMAL ANALYSIS

This section makes the guarantees of Truthprint precise. The results isolate what the design provides unconditionally (given sound primitives) from what remains an empirical question about semantic parsing, which is deferred to Section XI.

A. Carriers as Invariant-Equivalence Classes

Definition 1 (Invariant equivalence). *Let $I(\cdot)$ be the invariant extractor. Two structures G, G' are invariant-equivalent, written $G \sim_I G'$, iff $\text{InvariantEq}(I(G), I(G')) = 1$. For a discourse unit with invariants I , the admissible realization set is*

$$\mathcal{R}(I) = \{r : I(P(r)) = I \wedge Q(r) \geq \tau_Q\}. \quad (17)$$

A carrier alphabet A_j is a key-dependent partition of $\mathcal{R}(I)$ into classes; the encoder selects a class by symbol and the realizer emits any member. Thus the watermark ranges over the quotient $\mathcal{R}(I)/\sim_I$ rather than over tokens or embedding neighborhoods.

Assumption 1 (Validator soundness). *The hard-invariant validator V is sound: if V accepts a candidate r for invariants I , then $I(P(r)) = I$. (It need not be complete; rejected admissible candidates only reduce capacity, not fidelity.)*

Proposition 1 (Conditional protected-field fidelity). *Under Assumption 1, every committed output $y = R_\ell(E_K(x, m))$ satisfies $I(P(y)) = I(x)$.*

Proof sketch. Encoding commits a candidate only if V accepts it, and carriers with no accepted candidate are recorded as erasures rather than realized (Sec. VII-D). By soundness, acceptance implies invariant equality for that unit; taking the conjunction over committed units gives $I(P(y)) = I(x)$. $\square$

This is the structural property that logit-biasing methods cannot offer: they have no invariant checker in the loop, so quality preservation is measured post hoc rather than enforced.

a) *Empirical validator soundness.*: Assumption 1 is load-bearing, so we measure its empirical proxy—the validator’s *false-accept rate* on meaning-altering edits—rather than assume it silently. A false accept is a single-field tamper that the re-extracted invariants fail to flag. On the closed domain the typed contract detects every single-field tamper across all six protected fields (property C1, aggregate 1.000, CI [0.998, 1.000]: zero observed false accepts) while retaining attribution across benign paraphrases (0.990); the decode-only ablation accepts every tamper (property C2, 0.000), confirming that the invariant re-extraction—not the coding layer—is what supplies soundness. Under real NLLB translation the Stage-2 extractor still rejects meaning-altering edits at ≥ 0.99 per condition (Sec. X-F). These are empirical soundness measurements on the closed and machine-translated domains; a *wide-coverage* validator’s soundness remains open and is what the neural frontend of Appendix E must establish, so Proposition 1 stays conditional rather than absolute.

B. Unforgeability and False Positives

Theorem 1 (Fresh-payload unforgeability). *Model the tag as $t = \text{MAC}_K(m \parallel n \parallel h_I)$ truncated to τ bits. Any PPT adversary that causes authenticated attribution for a tuple (m, n, h_I) not previously tagged by the generator yields an EUF-CMA forgery against this truncated MAC with the same success probability (up to simulation overhead). Thus its success is bounded by the EUF-CMA advantage of the instantiated truncated MAC; guessing loss is already part of that advantage and is not added a second time.*

Proof sketch. Attribution requires a recovered payload whose tag equals $\text{MAC}_K(m \parallel n \parallel h_I)$ on a triple not produced by the generator. The reduction answers authorized generation requests with its tagging oracle and, when the detector accepts a fresh tuple, outputs that tuple and recovered tag as its forgery. Replay of a previously tagged tuple is excluded by freshness and must instead be handled by deployment policy. $\square$

Theorem 2 (Authenticated acceptance under an ideal-PRF model). *Assume the truncated MAC is a pseudorandom function on a fresh input and the decoded candidate tag is independent of its output. For a fixed (key, nonce) test, an independently produced candidate payload passes the τ -bit tag check with probability $2^{-\tau}$, up to the PRF distinguishing advantage. For Q tested key–nonce pairs, the union bound is $Q2^{-\tau}$ plus that advantage.*

This statement concerns only a candidate that reaches tag verification. For q distinct fresh candidates whose proposed tags are independent of ideal-PRF outputs, the union bound is $q2^{-\tau}$; adaptivity or repeated inputs must be covered by the concrete security game rather than assumed away. It is not an end-to-end detector FPR bound: parsing, candidate search, replay, carrier dependence, threshold calibration, and the number of tested keys and nonces all affect system-level error. Replayed generator outputs are also not fresh forgeries and require a nonce registry or provenance ledger. The per-test bound is a design constant, not a tuned threshold: at $\tau = 32$ it is 2.3×10^{-10} . The 0/20000 observation in Section X

is consistent with that bound but does not estimate such a small probability; its approximate 95% binomial upper bound is 1.5×10^{-4} . When full payload recovery fails, the fallback statistic of Eq. (16) is used. Under the null hypothesis of no watermark, the keyed match count over n reliable independent binary carriers is $k \sim \text{Binomial}(n, \frac{1}{2})$, so by Hoeffding’s inequality $\Pr[z \geq t \mid H_0] \leq e^{-t^2/2}$, giving a calibrated operating point. Dependence among carriers is an idealization that production calibration must correct (Sec. VIII).

C. Capacity and Minimum Reliable Length

Model each carrier j as a q_j -ary symbol on an erasure channel with erasure probability ϵ_j , where ϵ_j is the chance that a transformation destroys or obscures that site. The expected recoverable information is

$$\mathbb{E}[\text{bits}] \approx \sum_j (1 - \epsilon_j) \log_2 q_j. \quad (18)$$

With a rate- R code protecting an authenticated payload of $|p| = \mu + \tau$ bits, recovery succeeds with high probability once the surviving capacity exceeds $|p|$ with margin. For N uniform binary carriers ($q_j = 2$) with erasure rate ϵ and a systematic rate- R code ($k = RN$), this requires

$$(1 - \epsilon)N \gtrsim k \iff \epsilon \lesssim 1 - R. \quad (19)$$

Proposition 2 (Erasure recovery condition). *For a fixed linear $[N, k]$ code, erasure decoding succeeds exactly when the generator-matrix rows corresponding to the surviving positions have rank k . Consequently at least k positions must survive, giving the necessary bound $\epsilon \leq 1 - R$ for a realized erasure pattern. Whether recovery occurs near that boundary depends on the code construction and erasure distribution; the reference code’s finite-length transition is measured rather than claimed as a universal threshold.*

Equation (19) yields a necessary carrier-count condition $N \geq \lceil (\mu + \tau) / ((1 - \epsilon) \log_2 \bar{q}) \rceil$ for average carrier arity $\bar{q}$, together with the code-design constraint $R \leq 1 - \epsilon$. This is a planning bound, not a guarantee for a particular finite code. It explains why short or heavily summarized text can fall below capacity; the transition in Table III is specific to the implemented code and seeded erasure experiment.

D. Complexity

Let a document have S discourse units, $|C|$ carriers, κ candidates per carrier, and N code positions. Encoding cost is $O(S \cdot |C| \cdot \kappa \cdot c_{\text{val}})$, dominated by the validator cost c_{val} (a parse plus entailment check). Detection cost is $O(S \cdot c_{\text{parse}} + N \cdot c_{\text{map}} + D_{\text{ECC}})$, where the keyed map c_{map} is one HMAC evaluation and $D_{\text{ECC}} = O(Nk)$ for GF(2) erasure elimination (worst case $O(Nk^2)$). The cryptographic and coding operations scale with the number of carriers and are independent of the generator parameter count. End-to-end latency is nevertheless unknown because candidate generation, reparsing, and validation dominate and may themselves invoke large models; it must be measured for each implementation.

TABLE III

REFERENCE-IMPLEMENTATION DECODE SUCCESS VS. CARRIER ERASURE RATE ([96, 48] GF(2) CODE, 500 TRIALS PER POINT, SEEDED). FINE RESOLUTION NEAR THE CLIFF REVEALS A GRADUAL TRANSITION FROM FULL RECOVERY AT $\epsilon = 0.42$ TO COLLAPSE AT $\epsilon \geq 0.52$, AS PREDICTED BY PROPOSITION 2.

Carrier erasure ϵ	Recovery rate
0.10	1.000
0.20	1.000
0.30	1.000
0.40	1.000
0.42	0.998
0.44	0.982
0.45	0.968
0.46	0.950
0.48	0.774
0.50	0.290
0.52	0.000
0.55	0.000
0.60	0.000

X. REFERENCE IMPLEMENTATION AND VERIFICATION

To make the guarantees above concrete and reproducible, we provide a self-contained reference implementation (`reference/truthprint_poc.py`, standard library only) of the language-independent core: the canonical invariant digest h_I , the authenticated payload $p = m \parallel \text{MAC}_K(\cdot)$, the invariant-bound keyed carrier map, and a systematic GF(2) code with erasure decoding. This deliberately excludes the semantic parser so that the cryptographic and coding properties can be validated in isolation from model-dependent NLP. It exercises four properties and reports the following verified behavior (message $\mu = 16$ bits, tag $\tau = 32$ bits, code $[N, k] = [96, 48]$):

- **P1 (round-trip recovery).** An invariant-preserving transform that re-realizes carriers and erases 28/96 sites still yields exact payload recovery and a verifying MAC.
- **P2 (invariant binding).** Flipping a locked field (polarity positive→negative) changes h_I , so the MAC fails and the document is *not* attributed, illustrating Proposition 1’s contract.
- **P3 (cryptographic false positives).** Over 20,000 independent documents, 0 were attributed. This is consistent with Theorem 2; statistically, the observation alone only gives an approximate 95% binomial upper bound of 1.5×10^{-4} .
- **P4 (no global rule).** For a fixed symbol, 43/96 $\approx N/2$ realized options differ between two documents whose invariants differ, confirming the map is bound to (K, h_I, n) .

Table III reports decode success versus carrier erasure rate for the implemented [96, 48] code and seeded random-erasure protocol: full recovery (≥ 0.998) through $\epsilon = 0.42$, followed by 0.982 at $\epsilon = 0.44$, 0.774 at $\epsilon = 0.48$, 0.290 at $\epsilon = 0.50$, and zero observed recovery at $\epsilon \geq 0.52$. The location and shape are finite-code measurements; only the necessary survivor-count boundary follows from Proposition 2. These values characterize the coding layer, not detection on model-generated text.

A. Closed-Domain Linguistic Round-Trip

The core PoC operates on abstract carrier bits. To validate the linguistic layer end to end, a second self-contained script

(`reference/truthprint_linguistic_demo.py`) realizes an authenticated payload across templated factual sentences using two invariant-preserving carriers—voice (active/passive) and time-phrase position—then rewrites, re-parses, and detects on actual sentence strings (the running example of Section V-D). On a 32-sentence document (64 carriers, code [64, 32], $\tau = 20$) it verifies:

- **L1 (invariant fidelity).** Every benign transform leaves the parsed agent, predicate, patient, and polarity unchanged.
- **L2 (round-trip recovery).** With 19/64 carriers erased by realization-canonicalizing rewrites, the exact payload is recovered and the MAC verifies.
- **L3 (tamper detection).** Negating a single fact changes the document invariant digest, so attribution correctly fails.

This is a Stage-1 closed-domain demonstration, not a general multilingual parser; it establishes that the contract, keyed carriers, and erasure-aware decoder compose correctly once realized as text, and it isolates the remaining open problem as robust wide-coverage semantic parsing (Section XI).

B. Field-Level Challenge Set and Component Ablation

To test—rather than assume—that a *typed* invariant contract catches what an embedding *similarity* judge misses, we extend the closed-domain frontend to six independently protected fields (polarity, quantity, temporal direction, modality, attribution, causation) and generate minimal-pair probes (`reference` is packaged as `truthprint.challenge`; regenerate with `scripts/eval_challenge.py`). For each field we compare a meaning-preserving realization change against a single-field meaning-altering edit, and judge each with three detectors that differ only in the admissibility rule: the full typed+MAC contract, a decode-only ablation that drops the h_I -bound MAC, and an embedding-similarity gate (bag-of-words cosine) *calibrated* to reject at most 5% of legitimate paraphrases. On 300 documents of 24 facts ([48, 24] code), with 95% confidence intervals (Wilson for rates, bootstrap for cosines) at a fixed seed, we measure:

- **C1 (typed tamper detection).** The typed+MAC contract detects every single-field tamper across all six fields (aggregate 1.000, CI [0.998, 1.000]) while retaining attribution across benign paraphrases (0.990, CI [0.971, 0.997]).
- **C2 (MAC is load-bearing).** The decode-only ablation detects *no* tamper (aggregate 0.000, CI [0.000, 0.002]): the carriers still recover, so the h_I -bound MAC—not the coding layer—is what resists a meaning-altering edit.
- **C3 (similarity cannot separate meaning).** The calibrated embedding gate catches almost no tamper (aggregate 0.012, CI [0.008, 0.018]), because a single-field edit is lexically smaller than a legitimate voice/time reflow: in 48.6% of cases (CI [0.463, 0.509]) the tamper is at least as cosine-similar to the source as the paraphrase is, so *no* similarity threshold can separate them.

This isolates the interaction the design relies on—typed invariants supply the decision an embedding metric cannot, and the MAC binding converts that decision into tamper resistance—on real strings. It remains a closed-domain result; the wide-coverage multilingual analogue requires the neural

frontend and human annotations specified in Section XI and the accompanying data templates.

C. Real-MT Pilot: An Honest Negative Result

To test the closed-domain frontend against *real* machine translation rather than a channel model, we ran a small end-to-end pilot: 4 documents of 16 templated sentences were translated with a real neural MT system (NLLB-200-distilled-600M) into Korean and Hindi and back-translated (round-trip), yielding 192 real translation outputs. Measuring objectively—by attempting to re-parse each translated sentence with the Stage-1 frontend, so that a carrier counts as recovered only if the frontend can actually read it—the frontend parsed 0/192 outputs (parse coverage 0.000, 95% CI [0.000, 0.020]), including the round-trip English, which the rigid template parser cannot read once a neural system rephrases it. Consequently no payload was recovered from real MT. This is a deliberately honest negative result: it confirms with real translations what Table I lists as “not established,” and it localizes the bottleneck precisely at wide-coverage semantic parsing rather than at the coding or cryptographic core (which Sections X and X-B validate). It also shows why a naive evaluation must measure carrier survival from the transformed text itself: an annotation that merely *asserts* carrier survival can report a spurious perfect score. The pilot harness and the objective measurement ship with the artifact.

D. Stage-2 Multilingual Invariant Extractor

The pilot’s negative result is a property of the rigid Stage-1 *surface* parser, not of the meaning layer. To test whether the typed invariants *themselves* survive real translation, we add a closed-domain, lexicon-based multilingual extractor (English, Korean, Hindi, Chinese, Arabic, and German) that reads a translated sentence and recovers the nine invariant fields directly. We evaluate it on 4,800 real NLLB translations spanning all six languages—English round-trip and direct translation into Korean, Hindi, Chinese, Arabic, and German (800 source sentences over 50 documents $\times$ six conditions). Where the Stage-1 parser recovered 0/192, the Stage-2 extractor recovers the meaning layer at high rates (aggregate over all conditions, Wilson 95% CI): temporal direction 0.997 [0.995, 0.998], agent 0.997, polarity 0.992, causation 0.975, quantity 0.974, attribution 0.969, predicate 0.960, modality 0.910, and patient 0.905 [0.896, 0.913]. Recovery is strong for Hindi, Korean, Chinese, and German (per-condition all-exact recovery 0.81–0.88) and weakest for Arabic (0.29), where the MT system renders entities most variably (Arabic patient recovery 0.50). Two residual error sources are honest and instructive: entities the MT system renders inconsistently (e.g. “config drift”, “cache miss”, and Arabic patient nouns), and round-trip translation that itself alters a field (e.g. “must not” $\rightarrow$ “may not”), which the extractor faithfully reports rather than masks. This is direct real-data evidence for the central hypothesis: meaning-layer fields survive machine translation and are machine-recoverable, whereas the surface carriers of Section X-C do not. Because the recovered fields determine the invariant digest, a meaning-altering edit to any recovered

field is detected across languages. It remains a closed-domain result—a wide-coverage neural semantic parser is the next step—but it moves multilingual invariant recovery from “not established” to a measured, positive finding on real translations. The extractor and this evaluation ship with the artifact (`scripts/eval_multilingual.py`).

E. Preliminary Neural-Frontend Pilot

To probe whether a *wide-coverage* neural parser recovers the invariants where the closed lexicon cannot, we ran a small pilot with an instruction LLM as a drop-in extractor (`truthprint.neural_parser`, same `extract(text, lang)` interface). Sixteen sentences—ten closed-domain templates plus six *open-domain* facts (news/science/policy) with out-of-vocabulary wording—were machine-translated into six conditions (English clean and round-trip, and Korean/Arabic/Hindi/Chinese) by a hosted open model, and each translation was re-parsed by the neural extractor (96 sentences; Wilson 95% CI). The head-to-head against the in-repo lexicon extractor on the same text shows the expected crossover: on the *closed template* domain the hand-built lexicon still leads (field recovery 0.970 vs. the neural 0.726), but on the *open domain* the lexicon degrades below the neural frontend (0.481 [0.428, 0.536] vs. 0.614 [0.560, 0.666]). By field, the neural parser is stronger on the typed relational fields (predicate 0.896, causation 0.948, attribution and modality 0.938, temporal direction 0.688) and weaker on open-string agent/patient (0.094/0.177) and quantity (0.500), where an exact match across translation is hardest; consequently its strict all-nine-field exact rate is low. This is a feasibility pilot, not the NLLB-scale evaluation: it is small, one open model served both translation and extraction (a self-consistency caveat), German was dropped for translation-format failures, and it is machine translation by an LLM rather than NLLB. It nonetheless gives direct evidence for the paper’s stated next step—a neural frontend generalizes to open-domain wording where the lexicon abstains. The pilot backend and scorer ship with the artifact (`scripts/eval_neural_parser.py`).

a) *Separated backends and a translator-independent frontend.* That pilot let one model both translate and read its own output, a self-consistency confound. We address it two ways. First, the neural harness now enforces *separate* translation and extraction models as a checked contract (`neural_parser.SeparatedPipeline` rejects a shared model id), and the GPU handoff instantiates it with two distinct model families, so the wide-coverage numbers are unconfounded. Second, we add a frontend that is *model-separated by construction*: it reads the cached real translations with a real multilingual sentence *embedding* (`model2vec`)—not the translating model—and classifies the categorical invariant fields by nearest prototype (`scripts/eval_separated_frontend.py`). On open-domain wording this translator-independent frontend recovers *temporal direction* at 0.944 (Wilson CI [0.819, 0.985]) where the closed lexicon abstains (0.000 [0.000, 0.096]): a semantic frontend separated from the translator reads a field the lexicon cannot. Honestly, a *static* embedding is insufficient for the

full contract—polarity and modality sit near chance, aggregate 0.59 against the lexicon’s 0.69 on open domain—because a whole-sentence static vector encodes topic more than a single logical field. This quantitatively motivates the instruction-LLM frontend on the separated-backend harness rather than a static embedding, while removing the self-consistency confound from the in-repo measurement.

F. Meaning-Based Provenance Authentication Survives Real Translation

Recovering the meaning layer lets us realize the paper’s actual objective—*verifying consistency with a participating keyed generator*—without a surface payload that translation would erase. The producer registers, per sentence, a keyed τ -bit tag over the canonical digest of a declared *invariant contract* (a chosen field subset), stored in the out-of-band provenance ledger (like the nonce). The verifier re-extracts the invariants from the transformed sentence (Stage-2) and authenticates iff the tag reproduces. Authentication thus succeeds exactly when the contract’s meaning is preserved and recovered; a meaning-altering edit changes a contract field and is rejected; an unrelated document matches only with probability $2^{-\tau}$. On 4,800 real NLLB translations across six languages (Wilson 95% CI, $\tau = 32$), a compact contract (polarity, quantity, temporal direction, attribution, predicate, agent) yields sentence-level authentication TPR of 0.968 (DE), 0.968 (KO), 0.955 (ZH), 0.946 (HI), 0.936 (round-trip), and 0.629 (AR); tamper rejection ≥ 0.99 ; and false positives consistent with the cryptographic bound (0.001–0.010). Aggregating to the document level (a document is attributed when at least half of its sentences authenticate) gives 1.000 attribution (95% CI [0.929, 1.000]) for five of six conditions and 0.920 for Arabic, over 50 documents. Widening the contract to all nine fields lowers TPR (e.g. 0.289 for AR) but authenticates more meaning: an explicit, measured fidelity/robustness trade-off. This is the translation-robust counterpart to the surface-carrier pilot’s 0/192: provenance verified on the meaning layer survives real machine translation. One caveat is essential and we make it quantitative. Because authentication reproduces a tag over an invariant *contract*, two genuinely distinct but low-entropy generations that share the same compact contract collide *before* the tag is tested: their contract digests are equal, so the tag matches deterministically. The achievable false-positive rate is therefore bounded below by the *contract-collision probability*—governed by the entropy of the chosen contract on the deployment text—not by τ . Measuring this floor on the closed-domain corpus (`scripts/eval_semantic_fp.py`) is revealing: the six-field contract admits only 192 distinct values (7.6 bits), giving a collision floor of 5.2×10^{-3} ($\approx 1/192$). This matches the 0.001–0.010 false positives measured above and is $\sim 2 \times 10^7$ times the cryptographic 2^{-32} bound. In other words, the meaning-digest scheme’s observed false positives are the contract-entropy floor, not the tag bound; widening to nine fields lowers the floor to $\sim 1.8 \times 10^{-4}$ at the cost of translation robustness. Open-domain and especially short or formulaic text has *lower* contract entropy and thus a *higher* floor, so estimating contract entropy per corpus is a deployment

TABLE IV

REAL HEAD-TO-HEAD ON REAL NLLB TRANSLATION. A REAL KGW TOKEN WATERMARK COLLAPSES UNDER TRANSLATION WHILE THE MEANING-DIGEST SCHEME SURVIVES. KGW IS AT A CALIBRATED 1% FPR; THE MEANING-DIGEST ROW IS AT ITS CONTRACT-COLLISION OPERATING POINT ($\sim 5 \times 10^{-3}$), SO THE TWO ROWS ARE NOT AT A MATCHED FPR AND THE CLEAN CELL (—) IS UNDEFINED FOR A LEDGER SCHEME WITH NO IN-TEXT SIGNAL. SECTION X-H (TABLE V) GIVES THE MATCHED-FPR COMPARISON, ALL METHODS AT 1% FPR, ADDING A REAL SEMANTIC BASELINE AND ROBUST HASHING. REPLACES THE DIAGNOSTIC SIMULATOR (APPENDIX A).

Method	clean	RT	KO	HI	ZH	AR	DE
KGW (token)	0.960	0.380	0.040	0.040	0.020	0.000	0.080
Truthprint (meaning)	—	0.941	0.958	0.980	0.922	0.502	0.948

prerequisite; the $2^{-\tau}$ bound applies only once contract entropy exceeds τ bits. It remains a closed-domain Stage-2 result; a wide-coverage semantic parser and more documents/languages are the next steps. The method and evaluation ship with the artifact (`scripts/eval_provenance.py`).

G. Real Baseline Comparison: Token Watermark vs. Meaning Digest

To compare against a *real* token-level watermark on the same pipeline—rather than the diagnostic simulator of Appendix A, which must not rank methods—we embed a self-contained KGW watermark [9] ($\gamma=0.25$, $\delta=2.0$) during generation by a real model via logit biasing, translate the output with the same real NLLB-200-distilled-600M system into the six conditions, and detect on the translated text. Each method’s threshold is calibrated at 1% FPR on its null (unwatermarked) population; the meaning-digest scheme is shown at its own measured operating point (the contract-collision floor of Sec. X-F, not a tunable threshold). Table IV reports the result. The token watermark is detectable on clean text (0.960) but its signal is destroyed by real translation (0.000–0.380 across the six conditions), whereas the meaning-digest scheme survives (0.502–0.980). This is the real-data counterpart to the surface-carrier pilot’s 0/192 (Sec. X-C) and confirms, with a real token watermark and real MT, the cross-lingual token-watermark removal reported by He et al. [20]. It remains a single token baseline (SynthID/SIR via an open watermarking toolkit are an optional extension), a small generator, a closed-domain source, and one MT system; Arabic is weakest for the meaning-digest scheme too (0.502). The harness and scorer ship with the artifact (`scripts/eval_baselines_real.py`).

Because Table IV reports KGW at a calibrated 1% FPR but the meaning-digest scheme at its contract-collision operating point, and because KGW is a *token* baseline, Section X-H adds a matched-FPR comparison (every method at 1% FPR) that also includes a real *semantic* baseline (SemStamp) and the robust-hashing family.

H. Matched-FPR Semantic and Robust-Hashing Baselines

To answer the matched-operating-point question and to compare against a real semantic—not only token—baseline, we run an in-repo head-to-head in which *every* method is calibrated at the *same* 1% FPR on a null (unrelated-content) population,

TABLE V

MATCHED-FPR HEAD-TO-HEAD ON CACHED REAL LLM-MT TRANSLATIONS: TPR AT A COMMON 1% FPR (300 BOOTSTRAP DOCUMENTS OF 16 SENTENCES, SEEDED; REAL MULTILINGUAL EMBEDDING). RT = ENGLISH ROUND-TRIP; KO/HI/ZH/AR = DIRECT TRANSLATION. BOTTOM: FRACTION OF SINGLE-FIELD MEANING-ALTERING EDITS CORRECTLY REJECTED. ONLY THE TYPED CONTRACT BOTH SURVIVES TRANSLATION AND REJECTS A TYPED TAMPER.

Method	RT	KO	HI	ZH	AR
Meaning-digest (typed)	1.000	1.000	1.000	1.000	1.000
SemStamp (embed. region)	1.000	0.937	0.617	0.460	0.377
SimHash (robust hash)	1.000	0.590	0.000	0.883	0.633
MinHash (robust hash)	1.000	0.000	0.000	0.000	0.000
Exact SHA-256	0.867	0.000	0.000	0.000	0.000
<i>Single-field tamper rejection (closed domain, $n=200$):</i>					
Meaning-digest		1.000	[0.981, 1.000]		
SimHash / MinHash		0.000	[0.000, 0.019]		
Exact SHA-256		1.000	(but 0.0 TPR under MT)		

on the same cached real LLM-MT translations, with a real multilingual sentence embedding (`model2vec`). Alongside SemStamp [15] we include the robust-hashing family the meaning-digest scheme is naturally compared to (Sec. II-J); an exact SHA-256 surface hash, a char-shingle MinHash [27], and an embedding SimHash [28]. Documents are 16-sentence bootstrap samples (300 per condition, seeded); Table V reports TPR at matched 1% FPR with Wilson 95% CIs.

Three findings follow. First, a real embedding-region watermark (SemStamp) survives English round-trip (1.000) and a near language (Korean 0.937) but degrades as the embedding buckets drift cross-lingually (Hindi 0.617, Chinese 0.460, Arabic 0.377), whereas the typed meaning-digest survives all six conditions (1.000 at matched FPR). Second, robust hashing behaves as its surface dependence predicts: the exact hash and token MinHash collapse under any cross-script translation (0.000), and the embedding SimHash is partial and unstable (0.000–0.883). Third—decisive for positioning against robust hashing—a single-field, meaning-altering edit is *rejected* by the typed contract (1.000, CI [0.981, 1.000]) but by *neither* similarity hash (SimHash and MinHash both 0.000): a near-duplicate fingerprint cannot see which typed field changed. The exact hash rejects every tamper (1.000) but also rejects every legitimate translation (0.000 TPR cross-lingually)—the classic robustness/sensitivity trade-off that the meaning-digest resolves by hashing a *typed invariant contract* rather than a surface or a similarity region. This is the real-baseline, matched-FPR counterpart to challenge result C3. It shares the closed-domain, small-substrate (16 base items) caveats and uses a static embedding: a stronger encoder or a wide-coverage frontend may shift the baseline numbers, but the qualitative ordering—typed contract > embedding region > token/surface hash under translation, and typed contract *alone* rejecting single-field tampers—follows from where each signal lives. The harness ships with the artifact (`scripts/eval_semantic_baselines.py`).

To respect the page budget, three supporting parts are placed in the appendices: full reproducibility and notation (Appendix A), the reference software architecture and staged

TABLE VI

EVALUATION MATRIX: EACH RESEARCH QUESTION MAPPED TO ITS DECISIVE COMPARISON, MANIPULATED FACTOR, AND PRIMARY OUTCOME MEASURE.

RQ	Decisive comparison	Primary measure
RQ1	Truthprint vs. unconstrained paraphrase watermark	Invariant error rate (IER), contradiction rate
RQ2	Truthprint vs. token/embedding baselines, clean text	TPR@{1,0.1,0.01}%FPR, ROC-AUC
RQ3	Pre- vs. post-translation (1-hop, round-trip)	Payload BER, partial-recovery rate
RQ4	Sentence- vs. paragraph-level paraphrase	TPR@1%FPR vs. edit distance
RQ5	Watermarked vs. unwatermarked generation	Quality delta, added latency
RQ6	Recovery vs. document length	Authenticated bits recovered
RQ7	Adaptive attacker with vs. without invariant constraint	ValidRemoval rate (Eq. (1))

prototype roadmap (Appendices C–E), and the extended evaluation protocol—baselines, languages, attacks, and metrics (Appendix H). The body retains the research questions and the decisive evaluation matrix.

XI. EXPERIMENTAL METHODOLOGY

A. Research Questions

We propose the following research questions:

- RQ1: Does Truthprint preserve immutable meaning better than unconstrained paraphrase-based watermarking?
- RQ2: What true-positive rate is achieved at fixed false-positive rates on clean text?
- RQ3: How much signal survives single-hop and round-trip translation?
- RQ4: How robust is Truthprint to sentence- and paragraph-level paraphrasing?
- RQ5: What quality and latency costs are introduced?
- RQ6: How many authenticated payload bits can be recovered as document length increases?
- RQ7: Can an adaptive attacker remove the watermark while preserving all locked invariants?

Table VI fixes, for each question, the single manipulated factor and the outcome that would confirm or falsify the corresponding hypothesis, so that results cannot be reported selectively.

B. Diagnostic Simulation (Not an End-to-End Benchmark)

A controlled Stage-1 simulator checks the evaluation code paths and illustrates, under explicitly stipulated channel parameters, how signal placement (token vs. meaning layer) reacts to surface replacement and carrier erasure. Because the “baselines” are method-inspired proxies run from a shared harness rather than the published systems, the simulator *must not be used to rank methods* and is not evidence of real-translation performance. To keep this diagnostic from being mistaken for a benchmark, its full protocol and outputs (Tables VII and VIII) are deferred to Appendix A; the measured results this paper relies on are the real-MT findings of Sections X–C–X–F and the coding-core verification of Section X.

XII. FALSIFIABLE HYPOTHESES

The following are hypotheses to be tested at scale with the neural frontend. All hypotheses remain unconfirmed until evaluated on wide-coverage model text with real transformation systems:

- H1: Truthprint will outperform token-level watermarking under translation because the carrier is reconstructed from language-independent semantics.
- H2: Explicit invariant validation will reduce errors in negation, quantity, time, causality, and attribution.
- H3: Truthprint may underperform token methods on very short text because semantic carrier capacity accumulates across sentences.
- H4: ECC and erasure-aware decoding will improve recovery under parsing errors and partial deletion.
- H5: Strong summarization will remain difficult because it removes propositions and their associated carriers.
- H6: Combining structured carriers with embedding-based fallback evidence will outperform either alone across diverse languages.

XIII. SECURITY AND DEPLOYMENT CONSIDERATIONS

Carrier mappings should depend on the secret key, invariant digest, document nonce, and carrier identity. This prevents simple global carrier rules and reduces forgery risk. Exact detector scores should be access controlled or coarsened to reduce oracle attacks. The detector should localize evidence by paragraph and avoid attributing a mixed document to one source.

The reference API supplies the nonce to the detector out of band; natural-language carrier recovery alone does not discover it. A deployment must therefore define an authenticated nonce-resolution mechanism, such as a bounded time-bucket search plus deployment identifier or a provenance-ledger lookup. Every tested key–nonce pair consumes false-positive budget and must be included in calibration. A ledger should reject duplicate nonces and distinguish exact replay from fresh attribution. Splicing requires finer-grained binding (for example, paragraph identifiers and a document Merkle root); without it, a valid paragraph proves only that the participating generator produced that paragraph, not the provenance of the surrounding document.

Watermarks should support transparency rather than automated punishment. Every detection report should include the tested key and scheme version, text length, number of reliable carriers, calibrated false-positive rate, language and attack assumptions, confidence, and MAC verification status. User identifiers, raw prompts, and exact timestamps should not be embedded.

A. Deployment Path and Graceful Degradation

Truthprint is designed to slot into a serving stack as a semantic realization pass rather than as a modification of the token sampler. Two integration modes are practical. In the *inline* mode, the generator emits a draft, the encoder re-realizes admissible discourse units to carry payload symbols,

and the validator gates each candidate; although the coding and cryptographic layers are independent of model size (Sec. IX), the unmeasured generation and validation overhead may be substantial. In the *asynchronous* mode, watermarking runs as a post-generation service, which decouples watermark latency from the user-facing generation latency at the cost of a short delay before provenance is finalized.

Because capacity is unevenly distributed across text (Eq. 19), a deployable system must degrade gracefully. Low-capacity spans—code, formulas, legal clauses, and verbatim quotations—admit few invariant-preserving realizations and are therefore poor carriers. For such spans Truthprint records erasures and can optionally fall back to a token-level watermark such as SynthID-Text, yielding a layered scheme intended to combine semantic-layer transformation tolerance with token-layer short-span coverage; both properties require end-to-end evaluation. The detector localizes evidence per paragraph, reports the number of reliable carriers alongside the calibrated false-positive rate, and abstains rather than guessing when the recovered capacity falls below the minimum reliable length. This makes the system’s competence explicit at detection time, which is the behavior an auditor or platform integrator requires.

XIV. ETHICAL CONSIDERATIONS

A provenance detector can be misused to accuse authors, censor content, or make automated disciplinary decisions. Truthprint therefore treats a positive result as evidence about a participating keyed generation system, not proof of AI authorship or of misconduct. Deployments should expose the tested scheme and key identifier, use independently calibrated error rates, preserve an appeal path, and avoid adverse decisions based on the detector alone. Absence of a mark is likewise not evidence of human authorship.

The design has dual-use risks: public carrier families can aid removal attacks, while secret or overly detailed payloads can enable tracking. Payloads should exclude personal identifiers, prompts, and precise timestamps; keys and nonce registries require access control, rotation, and audit. Evaluation text should be licensed or consented, human annotation should be compensated and documented, and language-specific error disparities should be reported rather than hidden in an aggregate score. The present artifact uses synthetic template facts, no personal data, and no human subjects; the real-MT pilot (Secs. X-C–X-F) translates those synthetic facts with a public open-weights model (NLLB-200-distilled-600M) and involves no personal data or human subjects. Any scaled evaluation that adds human invariant or factual-equivalence annotation must use licensed or consented text and compensated, documented annotation.

XV. LIMITATIONS

Multilingual semantic parsing remains imperfect. Not every proposition admits multiple safe realization alternatives. Highly constrained text has low carrier capacity. Some apparently stylistic alternations change emphasis or information structure. Translation can omit modality, evidentiality, subjects,

or discourse relations. Strong summarization can destroy both content and watermark evidence. Finally, candidate generation and semantic validation add computational overhead compared with direct token-level methods. The current artifact does not evaluate real generators, translators, multilingual parsers, human judgments, official baselines, replay defenses, or adaptive attacks; consequently it supports feasibility claims about the coding core only, not comparative robustness or deployment readiness.

XVI. CONCLUSION

Truthprint is a proposed language-neutral, typed semantic IR for provenance watermarking. Its key contribution is a machine-checkable separation between immutable meaning and controlled realization choices. A secret-keyed encoder maps authenticated payload symbols to admissible realization classes, and a detector recovers those symbols after translation or paraphrasing through semantic parsing, canonicalization, error correction, and MAC verification.

The method cannot guarantee detection after arbitrary rewriting or semantic destruction. It instead targets a narrower and defensible objective: preserving provenance evidence across transformations that retain a substantial portion of the original meaning. Its value must be established through multilingual experiments, calibrated false-positive analysis, strong adaptive attacks, semantic-integrity evaluation, and direct comparison with token-, embedding-, and structure-based baselines.

REFERENCES

- [1] S. Dathathri *et al.*, “Scalable watermarking for identifying large language model outputs,” *Nature*, vol. 634, pp. 818–823, 2024. <https://www.nature.com/articles/s41586-024-08025-4>
- [2] I. Shumailov, Z. Shumaylov, Y. Zhao, N. Papernot, R. Anderson, and Y. Gal, “AI models collapse when trained on recursively generated data,” *Nature*, vol. 631, pp. 755–759, 2024. <https://www.nature.com/articles/s41586-024-07566-y>
- [3] European Parliament and Council, “Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act),” Official Journal of the European Union, 2024. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>
- [4] A. Liu, L. Pan, Y. Lu, J. Li, X. Hu, X. Zhang, L. Wen, I. King, H. Xiong, and P. S. Yu, “A survey of text watermarking in the era of large language models,” *ACM Comput. Surv.*, vol. 57, no. 2, 2024. <https://dl.acm.org/doi/10.1145/3691626>
- [5] Google DeepMind, “SynthID-Text reference implementation,” GitHub repository, 2024. <https://github.com/google-deepmind/synthid-text>
- [6] Google, “SynthID: Tools for watermarking and detecting LLM-generated text,” Responsible GenAI Toolkit; describes the Transformers integration, accessed 2026. <https://ai.google.dev/responsible/docs/safeguards/synthid>
- [7] R. Omid, Y. Dong, and B. Wang, “On Google’s SynthID-Text LLM watermarking system: Theoretical analysis and empirical validation,” arXiv:2603.03410, 2026. <https://arxiv.org/abs/2603.03410>
- [8] X. Han, Q. Li, J. Ni, and M. Zulkernine, “Robustness assessment and enhancement of text watermarking for Google’s SynthID,” arXiv:2508.20228, 2025. <https://arxiv.org/abs/2508.20228>
- [9] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, and T. Goldstein, “A watermark for large language models,” in *Proc. ICML*, 2023. <https://arxiv.org/abs/2301.10226>
- [10] X. Zhao, P. Ananth, L. Li, and Y.-X. Wang, “Provable robust watermarking for AI-generated text,” in *Proc. ICLR*, 2024. <https://arxiv.org/abs/2306.17439>
- [11] R. Kudithipudi, J. Thickstun, T. Hashimoto, and P. Liang, “Robust distortion-free watermarks for language models,” *Trans. Mach. Learn. Res.*, 2024. <https://arxiv.org/abs/2307.15593>
- [12] M. Christ, S. Gunn, and O. Zamir, “Undetectable watermarks for language models,” in *Proc. COLT*, 2024. <https://arxiv.org/abs/2306.09194>
- [13] P. Fernandez, A. Chaffin, K. Tit, V. Chappelier, and T. Furon, “Three bricks to consolidate watermarks for large language models,” in *Proc. IEEE WIFS*, 2023. <https://arxiv.org/abs/2308.00113>
- [14] A. Liu, L. Pan, X. Hu, S. Li, L. Wen, I. King, and P. S. Yu, “An unforgeable publicly verifiable watermark for large language models,” in *Proc. ICLR*, 2024. <https://arxiv.org/abs/2307.16230>
- [15] A. B. Hou *et al.*, “SemStamp: A semantic watermark with paraphrastic robustness for text generation,” in *Proc. NAACL*, 2024. <https://arxiv.org/abs/2310.03991>
- [16] J. Ren, H. Xu, Y. Liu, Y. Cui, S. Wang, D. Yin, and J. Tang, “A robust semantics-based watermark for large language models against paraphrasing,” in *Findings of NAACL*, 2024. <https://aclanthology.org/2024.findings-naacl.40/>
- [17] A. Liu, L. Pan, X. Hu, S. Meng, and L. Wen, “A semantic invariant robust watermark for large language models,” in *Proc. ICLR*, 2024. <https://arxiv.org/abs/2310.06356>
- [18] Z. Ye *et al.*, “SWAN: Semantic watermarking with Abstract Meaning Representation,” in *Proc. ACL*, 2026. <https://arxiv.org/abs/2605.04305>
- [19] J. Schäfer, C. Pilaszewicz, and G. Wunder, “Robust text watermarking for large language models via dual semantic embeddings,” arXiv:2606.31602, 2026. <https://arxiv.org/abs/2606.31602>
- [20] Z. He, B. Zhou, H. Hao, A. Liu, X. Wang, Z. Tu, Z. Zhang, and R. Wang, “Can watermarks survive translation? On the cross-lingual consistency of text watermark for large language models,” in *Proc. ACL*, 2024. <https://aclanthology.org/2024.acl-long.226/>
- [21] W. Qu, D. Yin, Z. He, W. Zou, T. Tao, J. Jia, and J. Zhang, “Provably robust multi-bit watermarking for AI-generated text via error correction code,” arXiv:2401.16820, 2024. <https://arxiv.org/abs/2401.16820>
- [22] T. Gloaguen, N. Jovanović, R. Staab, and M. Vechev, “Black-box detection of language model watermarks,” arXiv:2405.20777, 2024. <https://arxiv.org/abs/2405.20777>
- [23] T. Sander, P. Fernandez, A. Durmus, M. Douze, and T. Furon, “Watermarking makes language models radioactive,” in *Proc. NeurIPS*, 2024. <https://arxiv.org/abs/2402.14904>
- [24] X. Yi, S. Zheng, L. Wang, X. Wang, and L. He, “Unified attacks to large language model watermarks: Spoofing and scrubbing in unauthorized knowledge distillation,” arXiv:2504.17480, 2025. <https://arxiv.org/abs/2504.17480>
- [25] Y. Cheng, V. S. Sadasivan, M. Saberi, S. Saha, and S. Feizi, “Adversarial paraphrasing: A universal attack for humanizing AI-generated text,” arXiv:2506.07001, 2025. <https://arxiv.org/abs/2506.07001>
- [26] L. Pan *et al.*, “MarkLLM: An open-source toolkit for LLM watermarking,” in *Proc. EMNLP (System Demonstrations)*, 2024. <https://arxiv.org/abs/2405.10051>
- [27] A. Z. Broder, “On the resemblance and containment of documents,” in *Proc. Compression and Complexity of Sequences*, 1997.
- [28] M. S. Charikar, “Similarity estimation techniques from rounding algorithms,” in *Proc. ACM STOC*, 2002.

APPENDIX

This appendix gives the full protocol and outputs of the controlled Stage-1 diagnostic summarized in the main text. It validates the simulator implementation only; it is not a neural or translation benchmark and must not be used to rank the named published systems.

Tables VII and VIII report simulator outputs for Truthprint and four baselines on a shared closed-domain Stage-1 testbed (500 documents, 32 sentences each). To keep the comparison apples-to-apples in the absence of a wide-coverage neural frontend, every method is evaluated under one identical meaning-preserving channel and one identical protocol; the baselines are method-inspired proxies for selected aspects of each published method’s signal placement, run from the same harness (`scripts/eval_baselines.py`), and abstract the neural frontend (LM sampler, sentence encoder, AMR parser) in exactly the way the Truthprint Stage-1 numbers do. This diagnostic checks code paths and illustrates hypotheses under explicitly chosen channel parameters. It is not evidence that any named method attains the reported performance on real

TABLE VIII
DIAGNOSTIC TRANSFORMED-CHANNEL SIMULATION (TPR AT 1% FPR)
($\tau_{\text{tok}}, \epsilon_{\text{inv}}$) PER LANGUAGE PAIR[†]: EN→KO (0.92, 0.30), EN→HI
(0.95, 0.35), KO→EN (0.90, 0.28). OUTPUTS ILLUSTRATE THE
STIPULATED CHANNEL EQUATIONS, NOT REAL TRANSLATION.

Method	EN→KO	EN→HI	KO→EN
KGW proxy	0.020	0.018	0.030
DEW	0.000	0.000	0.000
SemStamp	0.996	0.988	0.994
SWAN	1.000	1.000	1.000
Truthprint	1.000	0.972	0.998

[†]Controlled Stage-1 channel; τ_{tok} (token replacement) and ϵ_{inv} (meaning-carrier erasure) per translation event. Only the Truthprint proxy in this harness exercises authenticated recovery; this table does not evaluate baseline security.

TABLE VII
DIAGNOSTIC CLEAN-CHANNEL SIMULATION (500 DOCS $\times$ 32
SENTENCES; THRESHOLD AT 1% FPR)[†]. DETECTION SATURATES FOR ALL
METHODS ON CLEAN TEXT; THE DISCRIMINATING TRANSFORMED
SETTING APPEARS IN TABLE VIII.

Method	TPR@1%FPR	ROC-AUC	BLEU-1
KGW proxy	1.000	1.000	—
SemStamp	1.000	1.000	—
SWAN	1.000	1.000	—
Truthprint	1.000	1.000	0.84

[†]Controlled simulation only, not a neural or translation benchmark. BLEU-1 measured only for Truthprint (real sentences); baseline fluency needs each method’s neural generator. Truthprint additionally applies a 20-bit tag check; this is not a measured end-to-end FPR.

translation, and it must not be used to rank the methods. End-to-end evaluation with real generators, translation systems, parsers, official baseline implementations, held-out calibration data, and confidence intervals remains future work.

Shared channel. A single meaning-preserving transform is characterised by two physical quantities that all methods see identically: the fraction τ_{tok} of surface tokens the transform replaces (cross-script translation replaces almost all tokens) and the fraction ϵ_{inv} of meaning-layer carriers lost to parse normalisation (the quantity Truthprint’s own analysis uses). Each method’s survival is then *derived* from these two numbers through its own signal dependence, so the relative ordering is forced by where the signal lives, not tuned per method: a KGW-style token-identity green-list proxy survives at $(1 - \tau_{\text{tok}})^2$; an edit-aligned token watermark (DEW) at $(1 - \tau_{\text{tok}})$ up to an edit budget then collapses; a sentence-embedding region (SemStamp) flips near the LSH boundary at a rate $\propto \tau_{\text{tok}}$ but is meaning-dominant; and meaning-layer signals (SWAN, Truthprint) are unaffected by τ_{tok} , losing only the ϵ_{inv} parse fraction.

Protocol. For each method a decision threshold is calibrated at 1% FPR on null (un-watermarked) documents; TPR is measured at that threshold after the channel; ROC-AUC is computed from watermarked-vs-null score populations. Truthprint is run through its real linguistic codec on actual sentences ($\mu = 12$, $\tau = 20$, $[64, 32]$ GF(2) code); its quality is unigram BLEU between the watermarked and an unconstrained realization of the same facts. Baseline fluency/quality is not measured because it requires each method’s neural generator, absent from the Stage-1 abstraction.

Reading the tables. The clean setting saturates because the proxy generators deliberately select matching symbols and provide many carriers. Under the stipulated high-replacement setting, token proxies lose signal by construction while semantic proxies retain it according to the chosen erasure parameter. These outcomes validate the simulator implementation, not H1 and not any comparison among the named published systems. P2 and P3 separately exercise Truthprint’s authenticated coding core (Section X); this harness does not evaluate the security properties of the other systems.

This section pins every choice needed to reimplement the language-independent core exactly, so that the reference scripts and the paper agree symbol for symbol. Nothing here is left to interpretation; the two scripts in `reference/` realize precisely these definitions.

A. Primitive Instantiation

The reference implementation fixes the following concrete choices.

- **Invariant digest.** $h_I = \text{SHA-256}(\text{JSON}(I))$, where JSON is canonical: keys sorted, separators “,” and “:”, UTF-8 encoded. Canonicalization makes h_I order- and whitespace-independent.
- **Authentication.** $\text{MAC} = \text{HMAC-SHA256}$; the tag is the first τ bits of the MAC over $m \parallel n \parallel h_I$. The payload is $p = m \parallel \text{tag}$ (length $|p| = \mu + \tau$).
- **Keyed carrier map (binary carriers).** $\pi_j = \text{LSB}(\text{HMAC-SHA256}(K, h_I \parallel n \parallel \text{ID}_j))$ with ID_j the 4-byte big-endian carrier index. The realized option is $a_j^* = \pi_j \oplus c_j$ and the recovered symbol is $\hat{c}_j = \pi_j \oplus a_j^{\text{obs}}$.
- **Error-correcting code.** A systematic $[N, k]$ linear code over GF(2) with generator $G = [I_k \mid P]$, where P is drawn once from a fixed pseudo-random seed. Erasure decoding solves the surviving linear equations by Gaussian elimination over GF(2); it succeeds iff the retained rows have rank k (Proposition 2).
- **Detection decision.** Attribute iff erasure decoding returns some $\hat{p}$ and MAC verifies on $(\hat{n}, n, h_I)$; otherwise report the calibrated statistic z of Eq. (16) against a threshold τ_z without asserting authenticated attribution.

B. Notation-to-Code Map

Table IX maps paper symbols to identifiers in `reference/truthprint_poc.py`, removing any ambiguity between the formalism and the code.

C. Parameters, Determinism, and How to Run

The core script uses $\mu = 16$, $\tau = 32$, $[N, k] = [96, 48]$; the linguistic script uses $\mu = 12$, $\tau = 20$, $[N, k] = [64, 32]$ over 32 sentences with two carriers each. All randomness is seeded: message bits and erasure sampling use fixed seeds, and the code matrix P uses a fixed seed, so every reported number is reproducible. The document nonce n is freshly sampled per run and passed explicitly to the detector; recovery is invariant to its value, so success does not depend on the nonce. Both

TABLE IX
MAPPING BETWEEN PAPER NOTATION AND REFERENCE-IMPLEMENTATION IDENTIFIERS.

Symbol	Meaning	Code identifier
h_I	canonical invariant digest	canonical_invariant_digest
p	authenticated payload	authenticate_payload
MAC	tag verification	verify_payload
ECC	systematic GF(2) code	GF2Code.encode
decode	erasure decoding	GF2Code.decode_erasure
π_j	keyed carrier offset	keyed_bit
a_j^*	realized option	realize_option
$\hat{c}_j$	recovered symbol	recover_symbol
D_K	detector	detect_document

scripts return a non-zero exit code if any asserted property fails, so they double as regression tests. Reproduction requires only Python 3 (standard library only); the paper builds with pdfplatex via make. The exact commands are:

```
python3 reference/truthprint_poc.py          # P1
-P4
python3 reference/truthprint_linguistic_demo.py # L1
-L3
make                                          #
      build main.pdf
```

Expected results are those reported in Section X: P1–P4 hold, and L1–L3 hold on sentence strings.

The same primitives are also packaged as an installable Python distribution (code/ in the accompanying repository) with a unit-test suite and continuous integration. Its command-line interface reruns all properties (truthprint selftest) and regenerates Table III deterministically from a fixed seed with 500 trials per erasure rate (truthprint repro-table).

A prototype can use the following modular layout:

```
truthprint/
schema/          # entities, events, relations,
mutability
parser/          # multilingual parser and
canonicalizer
carriers/        # voice, packaging, discourse,
ordering
encoding/        # payload, keyed mapping, ECC
realization/     # candidate generation and ranking
validation/      # hard constraints, NLI, numbers,
quality
detection/       # extraction, decoding, statistics
attacks/         # translation, paraphrase, summary,
splice
experiments/     # datasets, metrics, runners
```

D. Core Data Model

Listing 4. Illustrative Python data model.

```
from enum import Enum
from pydantic import BaseModel, Field

class Mutability(str, Enum):
    LOCKED = "locked"
    CONDITIONAL = "conditional"
```

```
REALIZATION_ONLY = "realization_only"

class Event(BaseModel):
    id: str
    predicate: str
    roles: dict[str, str]
    polarity: str = "positive"
    modality: str = "asserted"
    time: dict = Field(default_factory=dict)
    quantity: dict | None = None
    attribution: str | None = None

class CarrierSite(BaseModel):
    carrier_id: str
    family: str
    target_node_ids: list[str]
    alternatives: list[str]
    recoverability: float
    quality_cost: float
```

E. Keyed Mapping Skeleton

Listing 5. Secret-keyed carrier assignment skeleton.

```
import hmac
from hashlib import sha256

def keyed_index(key, invariant_hash, nonce,
               carrier_id, symbol, option_count):
    msg = b"|".join([
        invariant_hash,
        nonce,
        carrier_id.encode(),
        symbol.to_bytes(4, "big")
    ])
    digest = hmac.new(key, msg, sha256).digest()
    return int.from_bytes(digest[:8], "big") %
    option_count
```

F. Stage 1: Closed-Domain Proof of Concept

Begin with technical or news-style factual statements. Support entities, events, agent/patient roles, polarity, number, time, and causality. Start with carriers that are easy to validate: active/passive voice, proposition order for independent facts, causal connective family, clause splitting/merging, and nominal/verbal realization.

G. Stage 2: Multilingual Parsing

Map English, Korean, and Hindi into one predicate and role inventory using a combination of multilingual structured extraction, dependency parsing, semantic-role labeling, named-entity normalization, numerical and temporal normalization, and multilingual embeddings as a fallback alignment mechanism.

H. Stage 3: Document-Level Coding

Introduce multi-ary carrier alphabets, BCH or Reed–Solomon codes, carrier interleaving, synchronization markers, order-independent anchors, partial payload recovery, and provenance database lookup.

The following enumerations complete the falsifiable protocol summarized in Section XI; they are the baselines, languages, attacks, and metrics a full evaluation must cover.

I. Baselines

Experiments should include unwatermarked decoding, a representative Red–Green token watermark, SynthID-Text, SemStamp, SemaMark, semantic-invariant watermarking, SWAN, DEW, Truthprint without invariant validation, Truthprint without ECC, embedding-only Truthprint, and structure-only Truthprint. Standardized baseline implementations from an open watermarking toolkit should be used where available to ensure comparability [26].

J. Languages and Domains

Recommended languages include English, Korean, Hindi, Chinese, Arabic, and German or French. Domains should include factual news, encyclopedic explanations, technical documentation, scientific summaries, policy-style prose, dialogue, and creative writing. Code, formulas, legal clauses, and direct quotations should be reported separately due to low semantic carrier capacity.

K. Attacks

The evaluation should cover Unicode and punctuation edits, spelling correction, direct translation, round-trip and two-

hop translation, lexical and syntactic paraphrasing, sentence splitting/merging, proposition reordering, deletion, insertion, splicing, summarization at multiple compression ratios, regeneration from bullet points, repeated detector-guided paraphrasing, and carrier-targeted rewriting.

L. Metrics

Detection metrics include ROC-AUC, true-positive rate at 1%, 0.1%, and 0.01% false-positive rates, false-negative rate, payload bit-error rate, complete and partial recovery rates, minimum reliable text length, and detector latency.

Semantic integrity metrics include entity consistency, role consistency, polarity accuracy, numerical accuracy, temporal consistency, causal-direction consistency, attribution consistency, bidirectional entailment, contradiction rate, and human factual-equivalence judgment.

A weighted invariant error rate can be defined as

$$\text{IER} = \frac{\sum_i w_i E_i}{\sum_i w_i}, \quad (20)$$

where critical categories such as polarity and numeric value receive larger weights or are treated as hard failures.